

## Resolución: Optimización del Presupuesto de Jardines mediante Programación Dinámica

Este documento presenta el diseño y el análisis detallado de los algoritmos de Programación Dinámica requeridos para resolver el problema de la optimización del presupuesto de construcción de jardines del ayuntamiento bajo restricciones presupuestarias y de tiempo.

### 1. Algoritmo para Obtener el Mayor Gasto Posible

Este problema se modela como una variante del Problema de la Mochila 0-1 (*0-1 Knapsack*).

- Disponemos de una capacidad de presupuesto  $P$  (entero positivo).
- Tenemos  $J$  emplazamientos (objetos), cada uno con un coste  $c_j$  para  $1 \leq j \leq J$ .
- Queremos elegir un subconjunto de emplazamientos tal que la suma de sus costes maximice el gasto total, sin superar el presupuesto  $P$ .

### Ecuación de Recurrencia de Programación Dinámica

Definimos  $M[i, w]$  como el mayor gasto que es posible realizar considerando únicamente los primeros  $i$  emplazamientos ( $0 \leq i \leq J$ ) y disponiendo de un presupuesto parcial de  $w$  ( $0 \leq w \leq P$ ):

$$M[i, w] = \begin{cases} 0 & \text{si } i = 0 \text{ o } w = 0 \\ M[i - 1, w] & \text{si } c_i > w \\ \max \left( M[i - 1, w], M[i - 1, w - c_i] + c_i \right) & \text{si } c_i \leq w \end{cases}$$

### Algoritmo en Pseudocódigo

```

Funcion CalcularMayorGasto(J, P, c)
Inicio
  // Crear la tabla de programación dinámica de tamaño (J + 1) x (P + 1)
  M <- Matriz de dimensiones (J + 1) x (P + 1) inicializada en 0

  // Rellenar la tabla paso a paso
  Para i <- 1 Hasta J Con Paso 1 Hacer
    Para w <- 1 Hasta P Con Paso 1 Hacer
      Si c[i] > w Entonces
        M[i, w] <- M[i - 1, w]
      Sino
        opcion_no_construir <- M[i - 1, w]
        opcion_construir <- M[i - 1, w - c[i]] + c[i]
        M[i, w] <- Maximo(opcion_no_construir, opcion_construir)
      FinSi
    FinPara
  FinPara

  // El resultado óptimo está en la última celda de la tabla
  Retornar M[J, P]

```

## 2. Algoritmo para Identificar los Emplazamientos Elegidos

Para obtener la lista de los emplazamientos concretos donde se construirán los jardines, debemos realizar un recorrido hacia atrás (backtracking sobre la matriz  $M$  ya calculada) partiendo desde la celda óptima  $M[J, P]$ .

### Algoritmo en Pseudocódigo

```

Funcion ObtenerEmplazamientosAfortunados(J, P, c, M)
Inicio
    afortunados <- Lista vacía
    w <- P

    // Reconstrucción del camino desde el último elemento hacia el primero
    Para i <- J Hasta 1 Con Paso -1 Hacer
        // Si el valor cambia respecto a la fila anterior, significa que
        // el jardín i fue seleccionado obligatoriamente
        Si M[i, w] != M[i - 1, w] Entonces
            Añadir i a afortunados
            w <- w - c[i] // Restamos el presupuesto consumido por el jardín i
        FinSi
    FinPara

    Retornar afortunados
FinFuncion

```

## 3. Costes Temporales y Espaciales

### Algoritmo 1: CalcularMayorGasto

- **Complejidad Temporal:**  $O(J \cdot P)$ . Contiene dos bucles anidados: el externo se ejecuta  $J$  veces (número de emplazamientos) y el interno  $P$  veces (presupuesto). En cada celda realizamos operaciones elementales de tiempo constante  $O(1)$ .
- **Complejidad Espacial:**  $O(J \cdot P)$  para almacenar la tabla bidimensional  $M$ . (Nota: Si solo se deseara conocer el valor máximo y no reconstruir la lista, la complejidad espacial se podría optimizar a  $O(P)$  guardando únicamente la fila actual y la anterior).

### Algoritmo 2: ObtenerEmplazamientosAfortunados

- **Complejidad Temporal:**  $O(J)$ . Realiza un único bucle que recorre las filas desde  $J$  hasta 1. En cada paso se efectúa una comparación elemental en tiempo constante  $O(1)$ .
- **Complejidad Espacial:**  $O(1)$  adicional (memoria de control) o a lo sumo  $O(J)$  si contabilizamos el espacio que ocupa la lista de salida en el peor escenario.

## 4. Modificación con Restricción Temporal Secuencial

## Nueva Restricción

Ahora, además del coste  $c_j$  y el presupuesto  $P$ , tenemos:

- $D$ : Días totales disponibles para realizar todas las construcciones.
- $d_j$ : El número de días que se tarda en construir el jardín en el emplazamiento  $j$ .
- Al ser **construcción secuencial**, la suma de los días de los jardines elegidos no puede superar  $D$ .

Esto transforma nuestro problema en un **Problema de la Mochila Bidimensional** (Mochila con dos restricciones independientes de capacidad: Presupuesto  $P$  y Días  $D$ ).

## Ecuación de Recurrencia Modificada

Definimos  $M_2[i, w, t]$  como el mayor gasto que es posible realizar considerando los primeros  $i$  emplazamientos, disponiendo de un presupuesto parcial de  $w$  y de un plazo restante de  $t$  días:

$$M_2[i, w, t] = \begin{cases} 0 & \text{si } i = 0 \text{ o } w = 0 \\ M_2[i - 1, w, t] & \text{si } c_i > w \text{ o } d_i > t \\ \max \left( M_2[i - 1, w, t], M_2[i - 1, w - c_i, t - d_i] + c_i \right) & \text{en caso contrario} \end{cases}$$

## Algoritmo Modificado en Pseudocódigo

```

Funcion CalcularMayorGastoConTiempo(J, P, D, c, d)
Inicio
    // Crear tabla tridimensional de tamaño (J + 1) x (P + 1) x (D + 1)
    M2 <- Tabla 3D de dimensiones (J + 1) x (P + 1) x (D + 1) inicializada en 0

    Para i <- 1 Hasta J Con Paso 1 Hacer
        Para w <- 1 Hasta P Con Paso 1 Hacer
            Para t <- 1 Hasta D Con Paso 1 Hacer
                // Si el jardín excede el presupuesto disponible o el límite de días
                Si c[i] > w o d[i] > t Entonces
                    M2[i, w, t] <- M2[i - 1, w, t]
                Sino
                    opcion_no_construir <- M2[i - 1, w, t]
                    opcion_construir <- M2[i - 1, w - c[i], t - d[i]] + c[i]
                    M2[i, w, t] <- Maximo(opcion_no_construir, opcion_construir)
            FinSi
        EndPara
    FinPara
    FinPara

    Retornar M2[J, P, D]
FinFuncion
  
```